Copilot

Invite

Yesterday

You said

I need very detail step by step on how to create 3 vms on aws. I need 1 windows, 1 sql server database, and 1 email server. The vms must have the ability to communicate with the each other. Include networking on same subnets and seccurity for each. Explain the reason for each step.

Copilot said

See my thinking

You're basically designing a tiny, self-contained network in AWS with three servers that can talk to each other safely. Let's walk it step by step and I'll explain *why* each thing matters as we go.

1. High-level design

Component	Purpose	Key Points
VPC	Private network in AWS	Single VPC, single subnet for all 3 VMs
Subnet	IP range inside VPC	Same subnet so all VMs are "neighbors"
Security groups	Virtual firewalls for instances	Separate SGs per role, allow only needed ports
EC2 instances	Your 3 VMs (Windows, SQL, Email)	Launched into same subnet
IAM & key pair	Access & login	Control who can manage and log in

Goal: All three VMs live in the same subnet inside one VPC, can communicate privately over internal IPs, and are protected by security groups that only allow the traffic you actually need.

2. Prepare the AWS environment

2.1 Create or choose an AWS account and region

1. **Sign in to AWS Console**
    - **Reason:** You need an account and a region (e.g., us-east-1) where all resources will live together.
2. **Pick a region (top-right of console)**

- **Reason:** Keeping everything in one region ensures low latency and simpler management. All three VMs must be in the same region to share the same VPC/subnet.

3. Create the network (VPC and subnet)

3.1 Create a VPC

1. **Go to:** VPC service → **Your VPCs** → **Create VPC**.
2. **Choose "VPC only"** (simple option).
3. **Set:**
    - **Name tag:** ProjectVPC
    - **IPv4 CIDR block:** 10.0.0.0/16
4. **Create VPC.**

Reason:

- The VPC is your private network in AWS.
- 10.0.0.0/16 gives you plenty of IP addresses for future growth.
- Everything (subnets, instances, security groups) will live inside this VPC, so they can communicate privately.

3.2 Create a subnet (shared by all three VMs)

1. **In VPC console:** Subnets → **Create subnet**.
2. **Settings:**
    - **VPC:** select ProjectVPC.
    - **Subnet name:** AppSubnet.
    - **Availability Zone:** pick one (e.g., us-east-1a).
    - **IPv4 CIDR block:** 10.0.1.0/24.
3. **Create subnet.**

Reason:

- The subnet is a slice of the VPC's IP space.
- Putting all three VMs in 10.0.1.0/24 means they're on the same Layer 3 network—so they can talk directly via private IPs without routing complexity.
- Same subnet simplifies troubleshooting and avoids cross-subnet routing rules.

3.3 (Optional but common) Attach an Internet Gateway

If you want RDP/SSH from the internet or outbound internet access:

1. **Create Internet Gateway:**
    - VPC → **Internet Gateways** → **Create internet gateway**.
    - Name: ProjectIGW.
    - **Attach to VPC:** ProjectVPC.
2. **Update route table:**
    - Route tables → find the main route table for ProjectVPC.
    - Edit routes:
        - Add route: **Destination:** 0.0.0.0/0, **Target:** ProjectIGW.
3. **Associate route table with** AppSubnet (if not already).

Reason:

- Internet Gateway + route 0.0.0.0/0 lets instances reach the internet and be reachable (if security groups allow).
- You need this for RDP to Windows, downloading updates, etc.
- If you want a fully private environment, you can skip this and use VPN/Direct Connect instead—but most small setups use an IGW.

4. Plan security groups (firewalls)

We'll create three security groups:

- **SG-Windows-App:** For the Windows VM (application / general server).
- **SG-SQL-DB:** For the SQL Server VM.
- **SG-Email:** For the email server VM.

All three will:

- Allow **all outbound** traffic (default).
- Restrict **inbound** to only what's needed.

4.1 Security group for Windows VM (SG-Windows-App)

1. **Go to:** EC2 → **Security Groups** → **Create security group**.
2. **Settings:**
    - **Name:** SG-Windows-App
    - **Description:** Security group for Windows app server
    - **VPC:** ProjectVPC

3. **Inbound rules (examples):**
    - **RDP (TCP 3389)** from your office IP:
        - Type: RDP
        - Port: 3389
        - Source: My IP (or a specific CIDR like 203.0.113.10/32)
    - **SQL access to DB server (if app connects to SQL):**
        - You don't need inbound SQL here; the Windows VM will be a *client* to SQL. So no SQL inbound on this SG.
    - **Optional HTTP/HTTPS (if you host a web app):**
        - Type: HTTP, Port 80, Source: 0.0.0.0/0 (or restricted CIDR).
        - Type: HTTPS, Port 443, Source: 0.0.0.0/0.

Reason:

- RDP is needed to manage the Windows server. Restricting it to your IP avoids exposing RDP to the world.
- Only open web ports if you actually host a website or API.
- Keeping inbound rules minimal reduces attack surface.

4.2 Security group for SQL Server VM (SG-SQL-DB)

1. **Create SG:**
    - Name: SG-SQL-DB
    - Description: Security group for SQL Server database
    - VPC: ProjectVPC
2. **Inbound rules:**
    - **SQL Server (TCP 1433)** from Windows app server:
        - Type: MS SQL
        - Port: 1433
        - Source: SG-Windows-App (select by security group ID).
    - **RDP (TCP 3389)** from your IP (for admin):
        - Type: RDP
        - Port: 3389

- Source: My IP or specific CIDR.

Reason:

- Only the Windows app server should talk to SQL on port 1433, not the whole internet. Using SG-to-SG reference means: “only instances in SG-Windows-App can reach SQL.”
- RDP for admin is again restricted to your IP to avoid broad exposure.

4.3 Security group for Email server (SG-Email)

1. **Create SG:**
    - Name: SG-Email
    - Description: Security group for email server
    - VPC: ProjectVPC
2. **Inbound rules (depends on role):**
    - **SMTP (TCP 25)**:
        - Type: SMTP
        - Port: 25
        - Source:
            - If internal-only mail: SG-Windows-App (so only your Windows server can send mail).
            - If public mail server: 0.0.0.0/0 (but this is risky; better to lock down and use proper spam protections).
    - **Submission (TCP 587)** (if clients submit mail):
        - Type: Custom TCP
        - Port: 587
        - Source: appropriate CIDR or SG.
    - **RDP or SSH for admin:**
        - If Windows email server: RDP 3389 from your IP.
        - If Linux email server: SSH 22 from your IP.

Reason:

- Email servers are frequent attack targets. Limiting who can connect on SMTP/587 is crucial.
- If this is just for internal app email (e.g., sending notifications), you can restrict SMTP to only the Windows app server’s SG.

- Admin access (RDP/SSH) should be locked to your IP.

5. Create key pair (for login)

1. **Go to:** EC2 → **Key pairs** → **Create key pair**.
2. **Name:** ProjectKeyPair.
3. **Type:** RSA.
4. **Private key format:** pem (for OpenSSH) or ppk (for PuTTY).
5. **Download and store securely.**

Reason:

- For Windows EC2, you use the key pair to decrypt the Administrator password.
- For Linux EC2, you use the key pair directly for SSH.
- Without a key pair, you can't log in to manage the servers.

6. Create the three EC2 instances (VMs)

All three will:

- Use **VPC:** ProjectVPC.
- Use **Subnet:** AppSubnet (10.0.1.0/24).
- Use **Auto-assign public IP:** Enabled (if you want direct internet access) or Disabled (if you'll use VPN/bastion).
- Use **Key pair:** ProjectKeyPair.

6.1 Windows VM (general app server)

1. **Go to:** EC2 → **Instances** → **Launch instance**.
2. **Name:** Windows-App-Server.
3. **AMI:** Choose a Windows Server AMI (e.g., Microsoft Windows Server 2022 Base).
4. **Instance type:** e.g., t3.medium (adjust to your needs).
5. **Key pair:** ProjectKeyPair.
6. **Network settings:**
    - **VPC:** ProjectVPC.
    - **Subnet:** AppSubnet.
    - **Auto-assign public IP:** Enable (if you want RDP from internet).
    - **Firewall (security group):** Select existing SG-Windows-App.

7. **Storage:** Default is fine for testing; adjust if you need more.
8. **Launch instance.**

Reason:

- This VM will act as your application server and possibly as a client to SQL and email.
- Putting it in AppSubnet with SG-Windows-App ensures it can reach SQL and email servers via internal IPs while being protected from unnecessary inbound traffic.

6.2 SQL Server VM

You can either:

- Use a **pre-built SQL Server AMI** from AWS Marketplace (easier), or
- Use Windows Server and install SQL Server yourself.

Let's assume a SQL Server AMI.

1. **Launch instance:**
    - Name: SQL-Server-DB.
    - AMI: Microsoft SQL Server 2019 on Windows Server (or similar).
    - Instance type: t3.medium or larger depending on workload.
    - Key pair: ProjectKeyPair.
2. **Network settings:**
    - VPC: ProjectVPC.
    - Subnet: AppSubnet.
    - Auto-assign public IP: Disable (recommended for DB; access via Windows app server or VPN).
    - Security group: SG-SQL-DB.
3. **Storage:**
    - Use separate EBS volume for data if you want best practice (e.g., one for OS, one for SQL data/logs).
4. **Launch instance.**

Reason:

- Database servers should generally not be exposed directly to the internet.
- Same subnet as Windows app server ensures low-latency internal communication.

- SG-SQL-DB only allows SQL traffic from SG-Windows-App and RDP from your IP, which is a strong security posture.

6.3 Email server VM

You can choose:

- **Windows-based email server** (e.g., hMailServer, Exchange in more complex setups), or
- **Linux-based email server** (Postfix, Exim, etc.).

Let's assume a Linux email server (common for SMTP).

1. **Launch instance:**
    - Name: Email-Server.
    - AMI: Amazon Linux 2 or Ubuntu Server.
    - Instance type: t3.small or similar.
    - Key pair: ProjectKeyPair.
2. **Network settings:**
    - VPC: ProjectVPC.
    - Subnet: AppSubnet.
    - Auto-assign public IP:
        - If you want it to send/receive internet email directly: Enable.
        - If internal-only: Disable.
    - Security group: SG-Email.
3. **Launch instance.**

Reason:

- Email server in the same subnet can receive SMTP connections from the Windows app server and send mail out.
- SG-Email controls who can talk to it on SMTP/587 and who can administer it via SSH/RDP.

7. Configure OS and applications on each VM

7.1 Windows app server configuration

1. **Get Administrator password:**
    - In EC2 console, select Windows-App-Server.
    - **Connect** → **RDP client** → **Get password**.

- Upload ProjectKeyPair private key to decrypt password.

2. **RDP into the server** using public IP (if enabled) or via VPN/bastion.
3. **Configure hostname and basic settings** (Windows updates, firewall, etc.).
4. **Install application(s)** that will:
    - Connect to SQL Server using its **private IP** (e.g., 10.0.1.20).
    - Use email server's **private IP** (e.g., 10.0.1.30) for SMTP.

Reason:

- Using private IPs keeps traffic inside the VPC, which is more secure and cheaper than public traffic.
- RDP is only allowed from your IP, so random internet hosts can't brute-force your server.

7.2 SQL Server configuration

1. **RDP into SQL Server VM** (using its private IP via Windows app server or public IP if you enabled one).
2. **Confirm SQL Server is running** and listening on port 1433.
3. **Configure SQL Server:**
    - Create databases, logins, and permissions.
    - Ensure SQL Server allows remote connections (it usually does on EC2 AMIs).
4. **Firewall inside Windows (Windows Defender Firewall):**
    - Ensure inbound rule for SQL Server (TCP 1433) exists.
    - Restrict to the Windows app server's private IP if you want extra OS-level control.

Reason:

- Even though the security group already restricts access, OS-level firewall adds another layer of defense.
- Proper SQL configuration ensures only authorized logins and databases are accessible.

7.3 Email server configuration

For a Linux email server (e.g., Postfix):

1. **SSH into Email-Server** using key pair:
    - ssh -i ProjectKeyPair.pem ec2-user@<public-ip> (Amazon Linux) or ubuntu@<public-ip> (Ubuntu).
2. **Install mail server software**:

- Example (Ubuntu):

bash

Copy

sudo apt update

sudo apt install postfix

3. **Configure Postfix**:
    - Set hostname.
    - Configure main.cf for:
        - myhostname, mydomain, myorigin.
        - Relay rules (if needed).
    - Decide if it will:
        - Send mail only (relay for your app), or
        - Receive mail from the internet (MX records, DNS, etc.).
4. **Test SMTP from Windows app server**:
    - Use a simple script or telnet to connect to email server's private IP on port 25 or 587.

Reason:

- Installing and configuring the mail server defines how your environment sends/receives email.
- Testing from the Windows app server confirms that security groups and subnet routing are correct.

8. Verify communication between the three VMs

8.1 Check internal connectivity (same subnet)

From each VM, ping the others by private IP:

- From Windows app server:
    - ping 10.0.1.<SQL-IP>
    - ping 10.0.1.<Email-IP>
- From SQL server:
    - ping 10.0.1.<Windows-IP>
- From Email server:
    - ping 10.0.1.<Windows-IP>

Reason:

- Same subnet means they should reach each other directly via private IPs.
- If ping fails, check:
    - OS firewalls (Windows Firewall, iptables).
    - Security groups (they allow outbound by default, but inbound might be too strict).

8.2 Test application-level connectivity

1. **Windows → SQL:**
    - Use SQL Server Management Studio or your app to connect:
        - Server: 10.0.1.<SQL-IP>,1433
    - If it fails:
        - Check SG-SQL-DB inbound rule from SG-Windows-App.
        - Check SQL Server configuration and Windows firewall.
2. **Windows → Email:**
    - Use a test script or mail client to send a test email via SMTP:
        - SMTP host: 10.0.1.<Email-IP>
        - Port: 25 or 587.
    - If it fails:
        - Check SG-Email inbound rule from SG-Windows-App.
        - Check mail server logs.

Reason:

- Verifying at the application layer ensures not just network connectivity, but that the services themselves are reachable and correctly configured.
- This confirms your security groups are allowing exactly the traffic you intended.

9. Security considerations and hardening

9.1 Limit public exposure

- **RDP/SSH:**
    - Always restrict to your IP or a small trusted CIDR.
- **SQL Server:**
    - Never expose port 1433 to 0.0.0.0/0.
- **SMTP:**

- If you must expose SMTP to the internet, consider:
  - Spam protection.
  - Rate limiting.
  - Proper DNS (SPF, DKIM, DMARC).

Reason:

- Publicly exposed ports are the main entry points for attacks.
- Restricting them drastically reduces risk.

9.2 Use IAM roles instead of hard-coded credentials

- Assign IAM roles to instances if they need to access other AWS services (S3, Secrets Manager, etc.).
- Avoid storing AWS keys on the servers.

Reason:

- IAM roles are more secure and easier to manage than static credentials.

9.3 Keep OS and applications updated

- Enable automatic updates or patch regularly.
- Monitor logs for unusual activity.

Reason:

- Unpatched systems are vulnerable to known exploits.
- Logs help you detect misconfigurations or attacks early.

10. Summary of why each major step matters

- **VPC & subnet:** Create a private, controlled network where all three VMs live and can talk via internal IPs.
- **Security groups:** Act as per-server firewalls, letting you precisely control who can talk to whom and on which ports.
- **Key pair & IAM:** Control who can log in and who can manage resources, keeping access limited and auditable.
- **EC2 instances in same subnet:** Ensure simple, direct communication without complex routing.
- **Service configuration (Windows, SQL, Email):** Turn raw VMs into functional servers that fulfill your app's needs.
- **Verification & hardening:** Confirm everything works as intended and is locked down to only necessary traffic.

If you want, next we can sketch exact example IPs and specific security group rules in a more “config sheet” style so you can copy them into AWS.

Edit in a page

Message Copilot